Modèle Vélib’

Maxime Saffon, Hugo Foulon, Léo Ribierre

Sommaire

1. Objectif du projet
2. Notre modèle
3. Analyse théorique pour un vélo
4. Simulation
5. Limites du modèles

L’objectif du projet

- Construire un modèle pour un système de vélo partagé type Vélib’

- D’écrire un code python pour estimer la probabilité qu’une station soit vide à l’état stationnaire.

- **Hypothèse simplificatrice :** On suppose que chaque station a une capacité infinie (i.e. elle peut accepter une infinité de vélos).

2. Le modèle

Le modèle – Les notations

- On note N le nombre total de vélos.
- On note J le nombre total de stations.
- On note (X(t), t ≥ 0) le processus qui représente le nombre de vélos par station et en trajet entre deux stations.

$$E := \{(\mathrm{n_{jk}})_{\mathrm{j,k}\in[1,J]} \mid \Sigma_{\mathrm{j},k} n_{jk} = N, n_{jk} \geq 0\}$$

- $\mathrm{n_{jk}}$ = nombre de vélos en train d'effectuer un trajet de la station j à la station k.
- $\mathrm{n_{jj}}$ = nombre de vélos à la station j

Le modèle – Pour chaque station j

- Les utilisateurs (cyclistes) arrivent à la station j selon un processus de Poisson d'intensité λ_j
- Quand un utilisateur arrive à la station j:
  - Si $n_{jj} > 0$ alors il prend un vélo
  - Si $n_{jj} = 0$ alors il repart sans vélo (le client est perdu)

  } Donc $1_{\{n_{jj}>0\}}$

- On note p_{jk} la probabilité qu'un client aille de la station j à la station k.
  - $p_{jk} > 0 \quad p_{jj} = 0 \quad \forall j, \Sigma_k p_{jk} = 1$
- Donc par amincissement d'un processus de Poisson, on a J-1 processus de Poisson d'intensité $\lambda_j \times p_{jk} \times 1_{\{n_{jj}>0\}}$

Le modèle – Pour une station j à k

- La durée d'un trajet à vélo d'une station j à une station k suit une loi exponentielle de paramètre μ_{jk}
- Mais pour faire le trajet il faut qu'un vélo soit dans l'état n_{jk} elle suit donc suit une loi exponentielle de paramètre $\mu_{jk} \times 1_{\{n_{jk}>0\}}$

- Notre modèle n'autorise donc qu'un seul vélo à bouger à la fois. On est bien dans le cadre de Kelly et Yudovina et des closed migration processes.

Le modèle – L'opérateur de transition

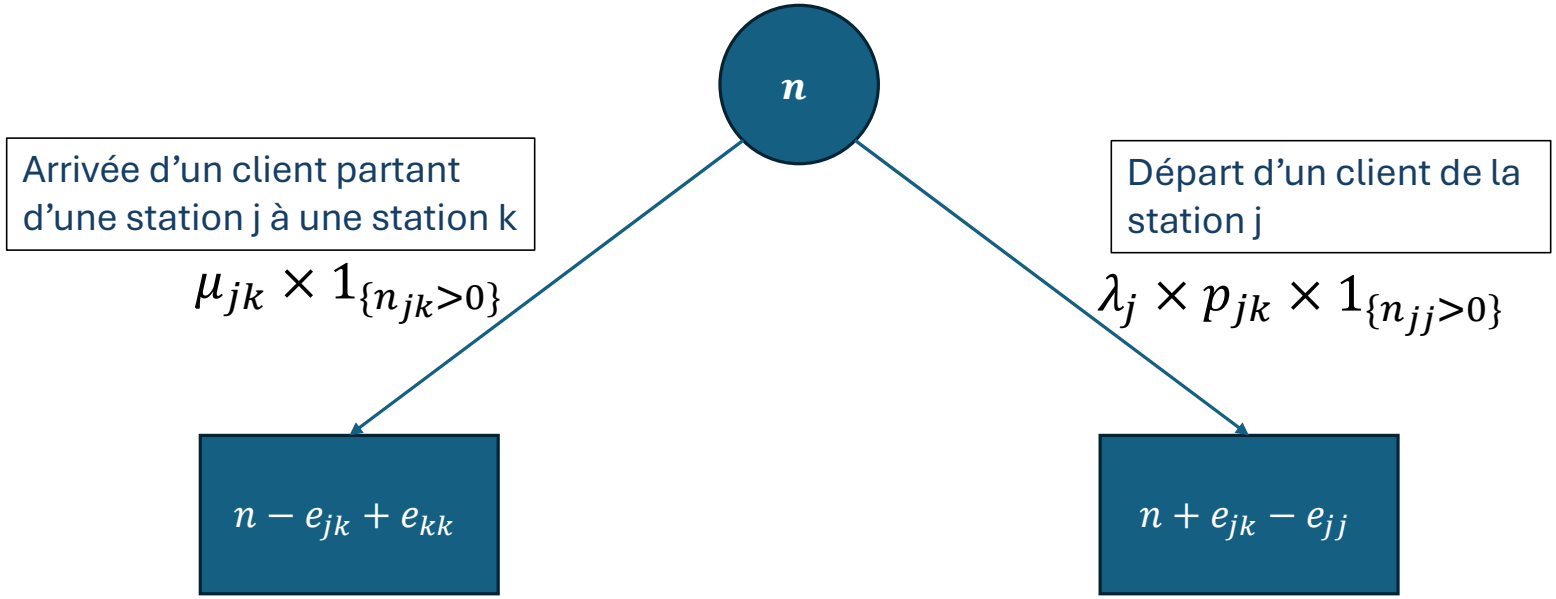
- On définit l'opérateur de transition T et les taux de transition associés:
  - Les fonctions de transitions T_{jk}^{D} qui représentent le départ d'un vélo de la station j à l'état « effectuant le trajet j vers k »:

$$T_{jk}^{D}(n) = n - e_{jj} + e_{jk} \qquad q\left(n, T_{jk}^{D}(n)\right) = \lambda_j \times p_{jk} \times 1_{\{n_{jj}>0\}}$$

  - Les fonctions de transition T_{jk}^{A} qui représente l'arrivée d'un vélo effectuant le trajet j vers k à la station k:

$$T_{jk}^{A}(n) = n - e_{jk} + e_{kk} \qquad q\left(n, T_{jk}^{A}(n)\right) = \mu_{jk} \times 1_{\{n_{jk}>0\}}$$

Le modèle – Schéma des transitions



Le modèle – les équations de traffic

- Pour calculer la distribution stationnaire on introduit des variables $(\alpha_{jk})_{jk}$
  - $\alpha_{jk} > 0 \qquad \Sigma_{jk}^{J} \alpha_{jk} = 1$
- On les utilise pour résoudre les traffics equations (flux entrant = flux sortant)
  - $\forall j, \alpha_{jj} \, \Sigma_{k \neq j}^{J} \mu_{kj} = \Sigma_{k \neq j}^{\alpha_{jk}} \lambda_j p_{jk}$
  - $\forall j \neq k, \alpha_{jk} \, \lambda_j \, p_{jk} = \alpha_{kk} \, \mu_{jk}$
- Puis on applique le théorème pour les closed migration processes:

$$\boxed{\pi(n) = G_N^{-1} \Pi_{j,k} \alpha_{jk}^{n_{jk}}}$$

3. Analyse théorique d’un vélo

Analyse théorique d'un vélo

$$
\begin{cases}
\mu_{12}\alpha_{12} = \lambda_1 P_{12}\alpha_{11}, \\
\mu_{13}\alpha_{13} = \lambda_1 P_{13}\alpha_{11}, \\
\mu_{14}\alpha_{14} = \lambda_1 P_{14}\alpha_{11}, \\
\mu_{15}\alpha_{15} = \lambda_1 P_{15}\alpha_{11}, \\
\mu_{21}\alpha_{21} = \lambda_2 P_{21}\alpha_{22}, \\
\mu_{23}\alpha_{23} = \lambda_2 P_{23}\alpha_{22}, \\
\mu_{24}\alpha_{24} = \lambda_2 P_{24}\alpha_{22}, \\
\mu_{25}\alpha_{25} = \lambda_2 P_{25}\alpha_{22}, \\
\mu_{31}\alpha_{31} = \lambda_3 P_{31}\alpha_{33}, \\
\mu_{32}\alpha_{32} = \lambda_3 P_{32}\alpha_{33}, \\
\mu_{34}\alpha_{34} = \lambda_3 P_{34}\alpha_{33}, \\
\mu_{35}\alpha_{35} = \lambda_3 P_{35}\alpha_{33}, \\
\mu_{41}\alpha_{41} = \lambda_4 P_{41}\alpha_{44}, \\
\mu_{42}\alpha_{42} = \lambda_4 P_{42}\alpha_{44}, \\
\mu_{43}\alpha_{43} = \lambda_4 P_{43}\alpha_{44}, \\
\mu_{45}\alpha_{45} = \lambda_4 P_{45}\alpha_{44}, \\
\mu_{51}\alpha_{51} = \lambda_5 P_{51}\alpha_{55}, \\
\mu_{52}\alpha_{52} = \lambda_5 P_{52}\alpha_{55}, \\
\mu_{53}\alpha_{53} = \lambda_5 P_{53}\alpha_{55}, \\
\mu_{54}\alpha_{54} = \lambda_5 P_{54}\alpha_{55},
\end{cases}
$$

$$
\begin{aligned}
\alpha_{11}(\lambda_1 P_{12} + \lambda_1 P_{13} + \lambda_1 P_{14} + \lambda_1 P_{15}) &= \mu_{21}\alpha_{21} + \mu_{31}\alpha_{31} + \mu_{41}\alpha_{41} + \mu_{51}\alpha_{51}, \\
\alpha_{22}(\lambda_2 P_{21} + \lambda_2 P_{23} + \lambda_2 P_{24} + \lambda_2 P_{25}) &= \mu_{12}\alpha_{12} + \mu_{32}\alpha_{32} + \mu_{42}\alpha_{42} + \mu_{52}\alpha_{52}, \\
\alpha_{33}(\lambda_3 P_{31} + \lambda_3 P_{32} + \lambda_3 P_{34} + \lambda_3 P_{35}) &= \mu_{13}\alpha_{13} + \mu_{23}\alpha_{23} + \mu_{43}\alpha_{43} + \mu_{53}\alpha_{53}, \\
\alpha_{44}(\lambda_4 P_{41} + \lambda_4 P_{42} + \lambda_4 P_{43} + \lambda_4 P_{45}) &= \mu_{14}\alpha_{14} + \mu_{24}\alpha_{24} + \mu_{34}\alpha_{34} + \mu_{54}\alpha_{54},
\end{aligned}
$$

$$
\left(\Sigma_{i=1}^{5} \Sigma_{j=1}^{5} \alpha_{ij} = 1 \right.
$$

```
A, b = build_A_b(lambda_, mu_, P)

x = np.linalg.solve(A, b) # Avec numpy
alpha = x.reshape(5, 5) # On délinéaris
```

Analyse théorique d'un vélo - Résultats

- $\pi(n_{jk}) = \frac{\alpha_{jk}}{G_5}$
- P(Station j soit vide à l'état stationnaire) $= 1 - \pi(n_{jj})$

P(station 3 soit vide à l'état stationnaire)	0.8245
P(station 4 soit vide à l'état stationnaire)	0.8445
P(station 5 soit vide à l'état stationnaire)	0.8655
P(station 6 soit vide à l'état stationnaire)	0.8443
P(station 7 soit vide à l'état stationnaire)	0.8407

4. Simulation

Simulation – Loi du temps de séjour

- Pour la simulation nous avons besoin de connaître le temps où X reste dans l'état n.
- X change d'état lorsque le premier évènement arrive (du schéma de transition) donc:

$$\min(\left(Exp\left(\mu_{jk}1_{\{n_{jk}\}}\right)\right)_{j\neq k}, \left(Exp\left(\lambda_j p_{jk}1_{\{n_{jj}>0\}}\right)\right)_{j\neq k})$$

- Donc le temps de séjour suit une loi exponentielle :

$$Exp(\Sigma_j\Sigma_{k\neq j}\mu_{jk}1_{\{n_{jk}>0\}} + \lambda_j p_{jk}1_{\{n_{jj}>0\}})$$

Simulation – Probabilité de transition

- Donc la probabilité d'aller à l'état $T_{jk}^{D}(n)$

$$\frac{\lambda_j p_{jk} 1_{\{n_{jj}>0\}}}{\Sigma_j \Sigma_{k\neq j} \mu_{jk} 1_{\{n_{jk}>0\}} + \lambda_j p_{jk} 1_{\{n_{jj}>0\}}}$$

- La probabilité d'aller à l'état $T_{jk}^{A}(n)$

$$\frac{\mu_{jk} 1_{\{n_{jk}>0\}}}{\Sigma_j \Sigma_{k\neq j} \mu_{jk} 1_{\{n_{jk}>0\}} + \lambda_j p_{jk} 1_{\{n_{jj}>0\}}}$$

Simulation – Probabilité stationnaire

- Ainsi pour avoir la distribution stationnaire π, il suffit pour chaque état de calculer le temps total passé dans cet état et de le diviser par la durée de la simulation soit:

$$\pi(n) = \frac{\text{Temps passé dans l'état n}}{\text{durée de la simulation}}$$

- Et pour avoir la probabilité stationnaire qu'une station soit vide:

$$\text{P(Station j soit vide à l'état stationnaire)} = \frac{\text{temps passé où pour chaque état n, } n_{jj} = 0}{\text{durée de la simulation}}$$

Simulation - Les données

- Matrice de routage → nous donne les p_{jk}
- Nombre de départ par minute → paramètre λ_j
- Nombre de vélo disponible → état initial des stations
- Nombre de vélo par route → état initial des vélos en train de faire un trajet
- Temps de traversée en minute → 1/data = μ_{jk}

Simulation – Le code

```python
class Model:
    def __init__(self, J, horizon, nb_velo=1):
        self.J = J # Le nombre de stations
        self.horizon = horizon # La durée de la simulation
        if nb_velo == 1:
            self.e = remplir_e() # L'état initiale
        else :
            self.e = remplir_e(filename1="data/nombre_de_velos_disponibles.csv",filename2="data/nombre_de_velos_par_routes.csv")
        self.mu_ = remplir_mu_() # Les valeurs des mu_jk
        self.P = remplir_P() # Les valeurs des p_jk
        self.lambda_ = remplir_lambda_() # Les valeurs de lamba_j
        self.proba_stat = defaultdict(int) # On instancie une table de hachage avec comme valeur de défault 0 pour les clés manquantes. Les clés de ce tableau sont
        les états e et la valeurs le temps de séjour dans chaque état pendant la simulation

    def indicatrice(self, A):
        if A:
            return 1
        return 0
```

Simulation – Fonctions auxiliaires

```python
def parametre_du_temps_de_sejour(self):
    '''
    Cette fonction renvoie le paramètre de temps de séjour selon l'état dans le quel nous sommes. Elle utilise la formule donnée juste avant.
    '''
    parametre = 0
    for j in range(self.J):
        for k in range(self.J):
            if j != k:
                parametre += self.mu_[j][k] * self.indicatrice(self.e[j][k] > 0)
                parametre += self.lambda_[j] * self.P[j][k] * self.indicatrice(self.e[j][j] > 0)
    return parametre

def T(self, i, j, k):
    '''
    Cette fonction représente la fonction de transition et modifie l'état du système.
    '''
    if i == 'D': # Si on est dans le cas d'un départ T^D
        self.e[j][k] += 1 # + e_jk
        self.e[j][j] -= 1 # - e_jj
    if i == 'A': # Si on est dans le cas d'une arrivé T^A
        self.e[k][k] += 1 # + e_kk
        self.e[j][k] -= 1 # - e_jk
```

Simulation – Fonction de transition

```python
def transition(self):
    '''
    Fonction qui calcule toutes les transitions possibles depuis l'état courant et leurs probabilités associées
    '''
    etat_suivant = [[],[],[]] # Premier tableau = un indice pour le deuxième tableau. Car quand on applique la fonction np.random.choice(_,p) cela ne marche
    seulement avec des int et non des tuples | Le deuxième tableau est l'état possible e.g ('A',j,k). Le troisième tableau = probabilité associée à
    transitionner dans cet état.
    i = 0
    for j in range(self.J):
        for k in range(self.J):
            # On parcour toutes les paires de stations (j,k)
            if k != j: # On exclue les trajets station à elle-même
                etat_suivant[0].append(i)
                etat_suivant[1].append(('A',j,k)) # Transition de type T^A
                etat_suivant[2].append((self.mu_[j][k] * self.indicatrice(self.e[j][k] > 0))/self.parametre_du_temps_de_sejour()) # Probabilité associée

                i += 1

                etat_suivant[0].append(i)
                etat_suivant[1].append(('D',j,k)) # Transition de type T^D
                etat_suivant[2].append((self.lambda_[j] * self.P[j][k] * self.indicatrice(self.e[j][j] > 0))/self.parametre_du_temps_de_sejour()) # Probabilité
                associée.

                i += 1

    return etat_suivant # On renvoie l'ensemble des états de transitions possible avec leur probabilités associée.
```

Simulation – La fonction principale

```python
def simulation(self):
    '''
    Cette fonction permet de faire tourner la simulation.
    '''
    temps = 0
    while temps < self.horizon:
        etat_courant = tuple(map(tuple, self.e)) # Une table de hachage ne peut pas avoir un tableau en clé donc on la transforme en tuple de tuple.
        duree = np.random.exponential(1/self.parametre_du_temps_de_sejour()) # temps de séjour dans l'état courant
        self.proba_stat[etat_courant] += duree # on ajoute dans la table de hachage le temps de séjour dans l'état courant
        temps += duree # on auguemente le temps avec le temps passé dans l'état courant.

        a = self.transition() # Calcul des transitions possibles et leurs probabilités.
        N = np.random.choice(a[0],p=a[2]) # Tirage aléatoire d'une transition avec a[0] les indices et a[2] les probabilités associés.
        N = a[1][N] # On la met sous la forme ('A' ou 'D', j, k) pour pouvoir appliquer la fonction de transition T
        self.T(N[0],N[1],N[2]) # On transitoinne d'état.
```

Simulation – Fonction de proba stationnaire

```python
def probabilite_stationnaire(self):
    '''
    Calcul la distribution stationnaire mais aussi renvoie la probabilité stationnaire qu'une station j soit vide.
    '''
    values = list(self.proba_stat.values()) # On récupère tous les durées de séjours dans chaque état.
    values = [x/self.horizon for x in values] # On divise par le temps de la simulation ce qui nous donne la distribution stationnaire approximative.

    stations = [0 for j in range(self.J)]

    for j in range(self.J): # Pour chaque station j
        for k in self.proba_stat.keys(): # On regarde chaque état e possible
            if k[j][j] == 0: # Si la station est vide à l'état e
                stations[j] += self.proba_stat[k]/self.horizon # Alors on rajoute le temps passé dans cet état au temps totale où la station j est vide

    # Affichage
    for j in range(self.J):
        print(f"P(Station {j} est vide à l'état stationnaire) = {stations[j]}")
```

Simulation - Résultats

	1 vélo calcul théorique	1 vélo : simulation	100 vélos : simulation
P(station 3 soit vide à l'état stationnaire)	0.8245	0.8210	0.0000
P(station 4 soit vide à l'état stationnaire)	0.8445	0.8451	0.0584
P(station 5 soit vide à l'état stationnaire)	0.8655	0.8657	0.1738
P(station 6 soit vide à l'état stationnaire)	0.8443	0.8443	0.0443
P(station 7 soit vide à l'état stationnaire)	0.8407	0.8446	0.0122

5. Les limites du modèle

Limites du modèle

- L’espace d’état est trop grand il pourrait être simplifié pour optimiser les calculs et donc le temps de calcul.

- Hypothèse simplificatrice loin de la réalité (on s’éloigne trop de la réalité pour rester dans le cadre des closed migration processes).